CSCE 411: Design and Analysis of Algorithms (Honors)

Lecture 2: Fast Fourier Transforms

Date: August 27, 2026 *Samson Zhou*

Course Logistics

- Read Chapter 30 of the textbook for today's material on the FFT.
- Syllabus quiz, HW, intro video due Fri, Sep 4.

1 Motivation: Polynomial Multiplication

We will introduce a powerful application of the divide and conquer paradigm: the Fast Fourier Transform (FFT). We will motivate it through the problem of polynomial multiplication.

Definition 1. A _____ in the variable x over an algebraic field is a representation of a function $A(x)$ as a formal sum:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

The values $a_0, a_1, \ldots, a_{n-1}$ are the _____ of the polynomial.

Example 1. Polynomial Multiplication

Input: Two polynomials $A(x) = \sum_{j=0}^{n-1} a_j x^j$ and $B(x) = \sum_{j=0}^{n-1} b_j x^j$ of degree bound n.

Output: A polynomial $C(x) = A(x)B(x)$ of degree bound $2n-1$ such that:

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j \quad \text{where} \quad c_j = \sum_{k=0}^{j} a_k b_{j-k}$$

An instance of this problem is multiplying $A(x) = 2 + x + 3x^2 + x^3$ and $B(x) = 1 + 2x + 4x^2 + x^3$.

Compute $C(x) = A(x) \cdot B(x)$, where $A(x) = 2+x+3x^2+x^3$ and $B(x) = 1+2x+4x^2+x^3$.

Notice that each coefficient $c_j = \sum_{k=0}^{j} a_k b_{j-k}$ is formed by multiplying pairs of coefficients whose indices sum to j, and adding the resulting products together. This operation is called a _____.

Question 1. What is the asymptotic runtime of multiplying two polynomials of degree bound n using the standard, naïve approach?

A $O(1)$

B $O(n)$

C $O(n \log n)$

D $O(n^2)$

E Other

Compute $C(1)$ for $C(x) = A(x) \cdot B(x)$, where $A(x) = 2 + x + 3x^2 + x^3$ and $B(x) = 1 + 2x + 4x^2 + x^3$.

2 Representations of Polynomials

To multiply polynomials faster, we can change how we represent them.

2.1 Coefficient Representation

We represent $A(x)$ as a vector of coefficients $a = (a_0, a_1, \ldots, a_{n-1})$.

- Evaluating $A(x)$ at a given point takes _____ time.
- Multiplying two polynomials takes _____ time.

2.2 Point-Value Representation

A polynomial of degree bound n is uniquely determined by its values at n distinct points. We can represent $A(x)$ as a set of n point-value pairs:

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

where $y_k = A(x_k)$ for $k = 0, 1, \ldots, n-1$.

Why use point-value representation?
If we have A and B in point-value form evaluated at the *same* points $x_0, \ldots, x_{2n-1}$, how long does it take to compute $C(x) = A(x)B(x)$ in point-value form?

Since _____, we just multiply the y-values! This takes _____ time!

Compute $C(1)$ for $C(x) = A(x) \cdot B(x)$, where $A(x) = 2 + x + 3x^2 + x^3$ and $B(x) = 1 + 2x + 4x^2 + x^3$, by computing and multiplying $A(1)$ and $B(1)$.

Question 2. Which of the following is the smallest number of point-value pairs that suffices to compute the polynomial $C(x) = A(x)B(x)$ uniquely, assuming A and B have degree bound n?

A n

B $2n$

C $n \log n$

D n^2

The Strategy:

- **Evaluate**: Convert A and B to point-value form at $2n$ points.
- **Point-wise Multiply**: Multiply all pairs of values in _____ total time.
- **Interpolate**: Convert C back to coefficient form.

Suppose we evaluate $A(x) = 2 + x + 3x^2 + x^3$ at $x = 1, 2, 3, \ldots, 10$.

$$\begin{bmatrix} A(1) \\ A(2) \\ A(3) \\ \vdots \\ A(10) \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{10} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 27 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & 10 & 100 & 1000 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 3 \\ 1 \end{bmatrix}$$

More generally, if we evaluate $A(x) = \sum_{j=0}^{n-1} a_j x^j$ at points $x_1, \ldots, x_n$, we are computing:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ 1 & x_3 & x_3^2 & \cdots & x_3^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

If we use arbitrary points, Evaluation and Interpolation take _____ time in total. We need to choose our evaluation points strategically!

3 The Divide-and-Conquer Strategy

We have seen that representing a polynomial by its values at n points makes multiplication easy, but converting between coefficient and point-value representations naively takes _____ time.

Our goal is therefore to speed up polynomial evaluation using _____.

Consider

$$A(x) = \sum_{j=0}^{n-1} a_j x^j.$$

Observe that we can separate the even and odd powers:

$$\begin{aligned} A(x) &= \sum_j a_{2j} x^{2j} + \sum_j a_{2j+1} x^{2j+1} \\ &= \sum_j a_{2j} (x^2)^j + x \sum_j a_{2j+1} (x^2)^j. \end{aligned}$$

Define the two polynomials

$$A_{\text{even}}(z) = \sum_j a_{2j} z^j, \qquad A_{\text{odd}}(z) = \sum_j a_{2j+1} z^j.$$

Then

This suggests a divide-and-conquer strategy. Suppose we want to evaluate A at n points

$$x_0, x_1, \ldots, x_{n-1}.$$

If the squared points

$$x_0^2, x_1^2, \ldots, x_{n-1}^2$$

contain only $n/2$ distinct values, then we only need to evaluate A_{even} and A_{odd} at $n/2$ points each.
Thus, evaluating A at n points reduces to:

- evaluating A_{even} at $n/2$ points;
- evaluating A_{odd} at $n/2$ points;
- combining the resulting values in $O(n)$ time.

This gives the recurrence

---

which solves to

So the key question is:

Can we find n distinct evaluation points whose squares contain only $n/2$ distinct values?

The answer will lead us to the **complex roots of unity**.

4 Background Math: Complex Roots of Unity

Our current goal is to find n distinct values whose squares contain only $n/2$ distinct values.

Let's see if we can find such a set of numbers by solving the equation $x^n = 1$ over the complex numbers. Let's try small powers of 2:

- $n = 1 : x^1 = 1 \implies x =$ ___
- $n = 2 : x^2 = 1 \implies x =$ _____
- $n = 4 : x^4 = 1 \implies x =$ _____
- $n = 8 : x^8 = 1 \implies x =$ _____

Notice an incredible pattern! The solutions for $n = 4$ contain the solutions for $n = 2$. Crucially, if we take our set of four solutions to $x^4 = 1$ and *square* them all, we get $\{1, 1, -1, -1\}$, which is exactly the set of two distinct solutions to $x^2 = 1$.

Algorithmic Takeaway: Squaring these points halves the number of distinct values! This is exactly the property we need to split our problem size in half for a divide-and-conquer algorithm.

Definition 2. The **n-th complex roots of unity** are the n complex numbers ω such that:

_____

They are given by the formula:

$$\omega_n^k = e^{2\pi i k/n} \quad \text{for } k = 0, 1, \ldots, n-1$$

The Unit Circle. In the complex plane, the _____ is the set of complex numbers whose distance from the origin is 1.

The _____ all lie on the unit circle, and form a regular polygon inscribed inside the unit circle.

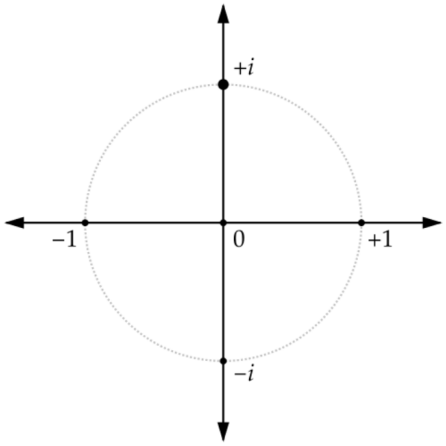

Figure 1: The complex roots of unity equally spaced around the unit circle.

To represent these points, we use **Euler's Formula**:

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

Furthermore, **De Moivre's Theorem** (which follows from standard exponent rules) states that exponentiating a complex number simply multiplies its angle:

$$(e^{i\theta})^m = e^{im\theta} = \cos(m\theta) + i\sin(m\theta)$$

This implies that taking powers of our roots of unity does not blow up to huge numbers; it simply *rotates* the value around the unit circle!

This formula gives us the coordinates of a point on the unit circle at angle θ. Since a full circle is 2π radians, we can generate all n distinct n-th roots of unity by taking evenly spaced angles: $\theta = 2\pi k/n$.

Lemma 4.1. ***Summation Lemma:*** *For any integer $n \geq 1$ and non-zero integer k not divisible by n:*

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = \_\_$$

5 Intuition for the Fast Fourier Transform

Recall $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$ needs to be evaluated at n different values of x. Suppose we evaluate $A_{\text{even}}(x^2)$ at

where ω is an n-th root of unity and n is a power of 2. Then

All we need to do is evaluate $A_{\text{even}}(x^2)$ at $\frac{n}{2}$ points!

To recover $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$ at $x = 1, \omega, \omega^2, \omega^3, \ldots, \omega^{n-1}$, also need to evaluate $A_{\text{odd}}(x^2)$ at $\frac{n}{2}$ points. Here is the pseudocode for the recursive FFT algorithm, which evaluates a polynomial at the n-th roots of unity.

Algorithm 1 Recursive-FFT(a)

$n = \textbf{length}(a)$
if $n == 1$ **then**
    Return a
end if
$\omega_n = e^{2\pi i/n}$
$\omega = 1$
$a^{[0]} = (a_0, a_2, \ldots, a_{n-2})$
$a^{[1]} = (a_1, a_3, \ldots, a_{n-1})$
$y^{[0]} = \text{RECURSIVE-FFT}(a^{[0]})$
$y^{[1]} = \text{RECURSIVE-FFT}(a^{[1]})$
Let y be a new array of length n
for $k = 0$ to $n/2 - 1$ **do**
    $y_k = y_k^{[0]} + \omega \cdot y_k^{[1]}$
    $y_{k+n/2} = y_k^{[0]} - \omega \cdot y_k^{[1]}$
    $\omega = \omega \cdot \omega_n$
end for
Return y

6 The Matrix Perspective (Discrete Fourier Transform)

Consider an example for $n = 8$, so ω is an eighth-root of unity, $\omega^8 = 1$.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ 1 & \omega^2 & \omega^4 & \omega^6 & \omega^8 & \omega^{10} & \omega^{12} & \omega^{14} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \omega^{12} & \omega^{15} & \omega^{18} & \omega^{21} \\ 1 & \omega^4 & \omega^8 & \omega^{12} & \omega^{16} & \omega^{20} & \omega^{24} & \omega^{28} \\ 1 & \omega^5 & \omega^{10} & \omega^{15} & \omega^{20} & \omega^{25} & \omega^{30} & \omega^{35} \\ 1 & \omega^6 & \omega^{12} & \omega^{18} & \omega^{24} & \omega^{30} & \omega^{36} & \omega^{42} \\ 1 & \omega^7 & \omega^{14} & \omega^{21} & \omega^{28} & \omega^{35} & \omega^{42} & \omega^{49} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix}.$$

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} = \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 & \omega^0 & \omega^2 & \omega^4 & \omega^6 \\ \omega^0 & \omega^3 & \omega^6 & \omega & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 & \omega^0 & \omega^4 & \omega^0 & \omega^4 \\ \omega^0 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega & \omega^6 & \omega^3 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 & \omega^0 & \omega^6 & \omega^4 & \omega^2 \\ \omega^0 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix}.$$

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} = \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^1 & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 & \omega^0 & \omega^2 & \omega^4 & \omega^6 \\ \omega^0 & \omega^3 & \omega^6 & \omega^1 & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 & \omega^0 & \omega^4 & \omega^0 & \omega^4 \\ \omega^0 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega^1 & \omega^6 & \omega^3 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 & \omega^0 & \omega^6 & \omega^4 & \omega^2 \\ \omega^0 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega^1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix}.$$

$$\begin{bmatrix} y_0 \\ y_2 \\ y_4 \\ y_6 \\ y_1 \\ y_3 \\ y_5 \\ y_7 \end{bmatrix} = \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 & \omega^1 & \omega^3 & \omega^5 & \omega^7 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 & \omega^2 & \omega^6 & \omega^2 & \omega^6 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 & \omega^3 & \omega^1 & \omega^7 & \omega^5 \\ \omega^0 & \omega^0 & \omega^0 & \omega^0 & \omega^4 & \omega^4 & \omega^4 & \omega^4 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 & \omega^5 & \omega^7 & \omega^1 & \omega^3 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 & \omega^6 & \omega^2 & \omega^6 & \omega^2 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 & \omega^7 & \omega^5 & \omega^3 & \omega^1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_2 \\ a_4 \\ a_6 \\ a_1 \\ a_3 \\ a_5 \\ a_7 \end{bmatrix}.$$

Four matrix multiplications:

$$\begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_2 \\ a_4 \\ a_6 \end{bmatrix} \qquad \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^1 & \omega^3 & \omega^5 & \omega^7 \\ \omega^2 & \omega^6 & \omega^2 & \omega^6 \\ \omega^3 & \omega^1 & \omega^7 & \omega^5 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \\ a_5 \\ a_7 \end{bmatrix}$$

$$\begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_2 \\ a_4 \\ a_6 \end{bmatrix} \qquad \begin{bmatrix} \omega^4 & \omega^4 & \omega^4 & \omega^4 \\ \omega^5 & \omega^7 & \omega^1 & \omega^3 \\ \omega^6 & \omega^2 & \omega^6 & \omega^2 \\ \omega^7 & \omega^5 & \omega^3 & \omega^1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \\ a_5 \\ a_7 \end{bmatrix}$$

Left two are the same. A single calculation suffices for the right two, because

$$\begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^1 & \omega^3 & \omega^5 & \omega^7 \\ \omega^2 & \omega^6 & \omega^2 & \omega^6 \\ \omega^3 & \omega^1 & \omega^7 & \omega^5 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \\ a_5 \\ a_7 \end{bmatrix} = \begin{bmatrix} \omega^0 & & & \\ & \omega^1 & & \\ & & \omega^2 & \\ & & & \omega^3 \end{bmatrix} \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \\ a_5 \\ a_7 \end{bmatrix}$$

$$\begin{bmatrix} \omega^4 & \omega^4 & \omega^4 & \omega^4 \\ \omega^5 & \omega^7 & \omega^1 & \omega^3 \\ \omega^6 & \omega^2 & \omega^6 & \omega^2 \\ \omega^7 & \omega^5 & \omega^3 & \omega^1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \\ a_5 \\ a_7 \end{bmatrix} = \begin{bmatrix} \omega^4 & & & \\ & \omega^5 & & \\ & & \omega^6 & \\ & & & \omega^7 \end{bmatrix} \begin{bmatrix} \omega^0 & \omega^0 & \omega^0 & \omega^0 \\ \omega^0 & \omega^2 & \omega^4 & \omega^6 \\ \omega^0 & \omega^4 & \omega^0 & \omega^4 \\ \omega^0 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_3 \\ a_5 \\ a_7. \end{bmatrix}$$

So, we only need two matrix-vector products!

This two-input, two-output computation is called a _____, named for the crossed-wing shape of its data-flow diagram. This allows the FFT to essentially run in place, i.e., $O(1)$ auxiliary space!

Runtime recurrence: $T(n) = 2T(n/2) + O(n)$

7 Summary (Discrete Fourier Transform)

When we evaluate a polynomial $A(x)$ at the n-th roots of unity, we are fundamentally applying a linear transformation to its coefficient vector. We can write this elegantly as a matrix-vector product $y = V_n a$, where a is the vector of coefficients, y is the vector of evaluations, and V_n is a special matrix called the **Vandermonde Matrix**.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \cdots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

Notice that the entry in row j and column k (0-indexed) is given by:

$$(V_n)_{j,k} = \_\_\_\_\_$$

This specific transformation is known as the **Discrete Fourier Transform (DFT)**.

8 Interpolation and the Inverse FFT

To convert our point-value results back into coefficient form, we need to invert this process. In matrix terms, we know $y = V_n a$. To solve for a, we must compute:

$$a = \_\_\_\_\_$$

Inverting an arbitrary $n \times n$ matrix takes $O(n^3)$ time. Fortunately, the Vandermonde matrix of the roots of unity has a beautifully simple inverse, which we can prove using the **Summation Lemma**.

Theorem 8.1. *The entries of the inverse matrix V_n^{-1} are given by:*

$$(V_n^{-1})_{j,k} = \frac{1}{n}\omega_n^{-jk}$$

Let's prove this! If this is the true inverse, then multiplying V_n^{-1} by V_n should yield the Identity matrix I_n.
Consider the entry in row j and column k of the product matrix $M = V_n^{-1}V_n$. This is

the dot product of the j-th row of V_n^{-1} and the k-th column of V_n:

$$M_{j,k} = \sum_{m=0}^{n-1} (V_n^{-1})_{j,m} (V_n)_{m,k} = \sum_{m=0}^{n-1} \left(\frac{1}{n}\omega_n^{-jm}\right)\left(\omega_n^{mk}\right) = \frac{1}{n}\sum_{m=0}^{n-1} \omega_n^{m(k-j)}$$

Case 1: On the Diagonal ($j = k$) If $j = k$, then $k - j = 0$. The exponent becomes 0, and $\omega_n^0 = 1$.

$$M_{j,j} = \frac{1}{n}\sum_{m=0}^{n-1} 1 = \frac{1}{n}(n) = \_\_$$

This correctly matches the 1s on the diagonal of the identity matrix!

Case 2: Off the Diagonal ($j \neq k$) If $j \neq k$, then $(k - j)$ is an integer strictly between $-n$ and n, and it is not zero. Thus, it is not divisible by n. By our **Summation Lemma**, the sum of these roots of unity is exactly ___!

$$M_{j,k} = \frac{1}{n}(0) = \_\_$$

This correctly matches the 0s off the diagonal of the identity matrix!

What does this mean for our algorithm?
Look closely at the resulting formula to compute the coefficients a_j:

$$a_j = \frac{1}{n}\sum_{k=0}^{n-1} y_k(\omega_n^{-1})^{jk}$$

This is *exactly* the same summation structure as the forward DFT! The only differences are swapping ω_n for ω_n^{-1} and dividing by n. This finally leads us to our main theorem.

9 Summary

We showed how to evaluate polynomials at roots of unity in $\Theta(n \log n)$ time. To finish polynomial multiplication, we need to interpolate the result back to coefficient form.

Interpolation. We just showed that interpolating at the complex roots of unity is identical to evaluating, but using ω_n^{-1} instead of ω_n, and dividing the final results by n. Thus, the **Inverse FFT** can also be computed in _____ time!

Fast Polynomial Multiplication. Given two polynomials $A(x)$ and $B(x)$ of degree bound n:

1. **Pad:** Add zeros to A and B so they have degree bound $2n$.

2. **Evaluate (FFT):** Compute point-value representations of A and B of length $2n$.
   Runtime: _____

3. **Point-wise Multiply:** Compute $y_k = A(\omega_{2n}^k)B(\omega_{2n}^k)$ for all k.
   Runtime: _____

4. **Interpolate (Inverse FFT):** Convert point-values y_k back to coefficients.
   Runtime: _____

Total Runtime for Polynomial Multiplication: _____.

By using the **divide-and-conquer** paradigm and evaluating at roots of unity, the FFT bypasses the naïve $O(n^2)$ approach! FFT is an incredibly important algorithm used heavily in signal processing, image compression, and fast large-integer multiplication!

10 Real-World Applications

The FFT is frequently cited as the most important numerical algorithm of the 20th century. By breaking the $O(n^2)$ barrier for convolution, it enabled the digital revolution.

1. Digital Signal Processing (DSP) In engineering, signals (like audio) are recorded as a sequence of amplitudes over time (the Time Domain). The FFT converts this signal into the Frequency Domain, where we can see which frequencies make up the sound.

- **Noise Filtering:** To remove static from a recording, one can take the FFT, zero-out the high-frequency components, and apply the Inverse FFT. Doing this naively takes $O(n^2)$, which is far too slow for real-time audio. FFT makes it possible.
- **Compression:** MP3 audio and JPEG images use variants of the FFT (like the Discrete Cosine Transform) to identify and discard frequencies that human ears and eyes cannot perceive, drastically reducing file sizes without noticeable quality loss.

2. Fast Integer Multiplication In modern cryptography (like RSA), computers must multiply prime numbers that are thousands of digits long. The grade-school multiplication algorithm takes $O(n^2)$ time.
However, we can treat a massive integer as a polynomial evaluated at $x = 10$! For instance:

$$1234 = 1(10^3) + 2(10^2) + 3(10^1) + 4(10^0)$$

The **Schonhage-Strassen Algorithm** converts huge numbers into polynomials, multiplies them using the FFT, and then processes the carries. This reduces large integer multiplication to $O(n \log n \log \log n)$ time, an essential speedup for encrypting data on the internet.

3. Number-Theoretic Transform (NTT) The FFT doesn't actually need complex numbers. We can do essentially the same divide-and-conquer algorithm over a finite field, using suitable roots of unity in place of the complex roots of unity. This is called the **Number-Theoretic Transform (NTT)**, and it gives us a fast way to compute convolutions using exact integer arithmetic. This is particularly useful in algorithms and cryptography, where floating-point rounding from the FFT can be undesirable.

4. Convolution Convolution itself is often a useful tool in algorithms. Recall that for two sequences $(a_0, \ldots, a_n)$ and $(b_0, \ldots, b_m)$, their convolution is the sequence $c_k = \sum_{i+j=k} a_i b_j$. A common situation is that we want to compute c_k for every possible k. Naïvely, this takes $O(nm)$ time, but FFT allows us to compute the entire convolution in $O((n+m)\log(n+m))$ time.

This also connects to *generating functions*. If we encode the sequences as polynomials $A(x) = \sum_i a_i x^i$ and $B(x) = \sum_j b_j x^j$, then the coefficients of $A(x)B(x)$ are exactly the convolution c_k. Thus, many counting problems that can be expressed using generating functions ultimately reduce to polynomial multiplication, and hence can sometimes be accelerated using FFT.

Example 2. Convolution

An engineer is testing two sensors. Each sensor can produce measurements at integer frequencies $0, 1, \ldots, n-1$. The first sensor produces a_i measurements at frequency i, while the second produces b_i measurements at frequency i.

For every possible frequency s, determine how many pairs of measurements, one from each sensor, have frequencies that add up to s. Compute the answer for every possible s.

For example, if the first sensor has 3 measurements at frequency 2, and the second has 4 measurements at frequency 5, then these contribute 12 pairs with total frequency 7.